Rajalakshmi Engineering College

Name: Harsan J
Email: 240701179@rajalakshmi.edu.in
Roll no: 240701179
Phone: 8925466195
Branch: REC
Department: CSE - Section 10
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 8_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Alice is designing a program that requires users to enter positive numbers. She wants to implement a solution that validates whether the entered number is positive. In case the input is not a positive number, she wants to throw a custom exception.

The number should be a positive integer.If this condition is violated, the program should throw a custom exception:InvalidPositiveNumberException with the message "Invalid input. Please enter a positive integer."

Implement a custom exception, InvalidPositiveNumberException , to handle cases where the entered number does not meet the specified criteria.

Input Format

The input consists of an integer value 'n', representing the entered number.

Output Format

The output is displayed in the following format:

If the validation passes, print

"Number {number} is positive."

The {number} represents the entered positive integer.

If the entered number is negative then it displays

"Error: Invalid input. Please enter a positive integer."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 100
Output: Number 100 is positive.

Answer

```
class InvalidPositiveNumberException extends Exception{
   public InvalidPositiveNumberException(String message)
{
      super(message);
}

}
class Main
{
   public static void main(String[] args)
{
      java.util.Scanner sc = new java.util.Scanner(System.in);
      int n = sc.nextInt();
      try
{
```

```
            validateNumber(n);
            System.out.println("Number " + n + " is positive.");
} catch (InvalidPositiveNumberException e)
{
            System.out.println("Error: " + e.getMessage());

}
        sc.close();
}
    public static void validateNumber(int n) throws
InvalidPositiveNumberException
{
        if (n <= 0)
{
            throw new InvalidPositiveNumberException("Invalid input. Please enter a
positive integer.");
}

}

}
```

Status : Correct **Marks : 10/10**

2. Problem Statement

Tim was tasked with creating a user profile system that validates the user's date of birth input. The system should throw a custom exception, InvalidDateOfBirthException, if the date is not in the specified format "dd-mm-yyyy" or if it represents an invalid calendar date.

The main method takes user input, validates the date of birth, and prints whether it is valid or not.

Input Format

The input consists of a string, representing the date of birth of the user.

Output Format

The output displays one of the following results:

If the entered date of birth is valid according to the specified format, the program prints:

"[Date] is a valid date of birth"

If the entered date of birth is not valid according to the specified format, the program prints:

"Invalid date: [Date]"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 01-01-2000

Output: 01-01-2000 is a valid date of birth

Answer

```
// You are using Java
import java.text.ParseException;
import java.text.SimpleDateFormat;
class InvalidDateOfBirthException extends Exception
{
   public InvalidDateOfBirthException(String message)
{
      super(message);
}

}
class Main
{
   public static void main(String[] args)
{
      java.util.Scanner sc = new java.util.Scanner(System.in);
      String dateInput = sc.nextLine().trim();
      try
{
         validateDateOfBirth(dateInput);
         System.out.println(dateInput + " is a valid date of birth");
} catch (InvalidDateOfBirthException e)
```

```
{
        System.out.println("Invalid date: " + dateInput);
}

      sc.close();

}
   public static void validateDateOfBirth(String date) throws
InvalidDateOfBirthException

{
      SimpleDateFormat sdf = new SimpleDateFormat("dd-MM-yyyy");
      sdf.setLenient(false);
      try
{
         sdf.parse(date);
} catch (ParseException e)

{
         throw new InvalidDateOfBirthException("Invalid date format or value");
}

}

}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Faustus is managing his bank account and wants to create a program to update his account balance based on certain conditions. However, he needs to handle specific scenarios related to invalid inputs and insufficient balances. Faustus wants to update his account balance. He inputs the current balance and the amount to be updated.

The initial account balance should be positive. If Faustus enters a negative initial balance, the program should throw an InvalidAmountException with the message "Invalid amount. Please enter a positive initial balance."If the

amount to be updated is negative, the program should check if the subtraction results in a negative balance. If so, it should throw an InsufficientBalanceException with the message "Insufficient balance."If the amount to be updated is positive, it should be added to the current balance, and the new balance should be printed.

Implement a custom exception, InvalidAmountException, and InsufficientBalanceException, to manage his bank account.

Input Format

The first line of input consists of a double value 'd', representing the initial account balance.

The second line of input consists of a double value 'd1', representing the amount to be updated.

Output Format

The output is displayed in the following format:

If the validation passes, print

"Account balance updated successfully! New balance: {new_balance}"

where {new_balance} is the updated account balance.

If the initial bank amount is negative it displays

"Error: Invalid amount. Please enter a positive initial balance."

If the updated amount exceeds the initial account balance in withdrawal it displays

"Error: Insufficient balance."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1000

500
Output: Account balance updated successfully! New balance: 1500.0

Answer

```
// You are using Java
import java.util.Scanner;
class InvalidAmountException extends Exception
{
   public InvalidAmountException(String message)
{
      super(message);
}

}
class InsufficientBalanceException extends Exception
{
   public InsufficientBalanceException(String message)
{
      super(message);
}

}
public class Main
{
   public static void main(String[] args)
{
      Scanner sc = new Scanner(System.in);

      double balance = sc.nextDouble();
      double updateAmount = sc.nextDouble();

      try
{
         if (balance < 0)
{
            throw new InvalidAmountException("Invalid amount. Please enter a
positive initial balance.");
}

         if (updateAmount < 0)
{
            if (balance + updateAmount < 0)
```

```
{
                throw new InsufficientBalanceException("Insufficient balance.");
} else
{
                balance += updateAmount;
}

} else

{
              balance += updateAmount;
}
          System.out.printf("Account balance updated successfully! New balance:
%.1f%n", balance);

} catch (InvalidAmountException e)

{
          System.out.println("Error: " + e.getMessage());


} catch (InsufficientBalanceException e)

{
          System.out.println("Error: " + e.getMessage());

} finally
{

          sc.close();


}


}

}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

A company is developing a user registration system that requires users to provide valid email addresses. The development team is implementing an EmailValidator program to ensure that the entered email addresses meet certain criteria using exception handling.

The email address must contain the "@" symbol.The email address must consist of a non-empty username(before "@" symbol) and a non-empty domain(after "'@" symbol).The domain part of the email address must contain at least one period (".").The email address must not contain leading or trailing spaces.

Implement a custom exception, InvalidEmailException, to fulfill the company's requirements and validate it according to the specified rules.

Input Format

The input consists of a string value 's', which represents the email address.

Output Format

The output is displayed in the following format:

If the entered email address is valid according to the specified rules, the program prints:

"Email address is valid!"

If the entered email address misses the username or domain part or misses "@" symbol or has two or more "@" symbols or misses '.' in the domain part it outputs:

"Error: Invalid email format."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: johndoe@example.com

Output: Email address is valid!

Answer

```
// You are using Java
import java.util.Scanner;
class InvalidEmailException extends Exception
{
   public InvalidEmailException(String s)
{
      super(s);
}
}
class main
{
   public static void check(String s) throws InvalidEmailException
{
     if(!s.endsWith(".com") || !s.contains("@"))
{
        throw new InvalidEmailException("Invalid email format.");
}
     String[] p=s.split("@");
     if(p.length!=2 || p[0].isEmpty()||p[1].isEmpty())
{
       throw new InvalidEmailException("Invalid email format.");
}
     System.out.print("Email address is valid!");
}
   public static void main(String[] args)
{
     Scanner sc=new Scanner(System.in);
     String s=sc.nextLine();
     try
{
        check(s);
}
     catch(InvalidEmailException e)
{


        System.out.print("Error: "+e.getMessage());
```

```
}

}

}
```

Status : Correct *Marks : 10/10*